Operation fsoc

Case Study of a Defense-First Security Engineering Project

Gurmann Basran

Prepared 2026-04-16. Updated as the project progresses.

1 What this is

Operation fsoc is the name I gave to the multi-phase security engineering project I have been running out of my own homelab since August 2025. It started as a homelab upgrade and grew into something more useful: an operational infrastructure for an emerging cybersecurity professional, built from scratch on a student budget, designed so that every phase teaches me a domain of the field while also locking down a real system I use every day.

This document is the public face of the project. It is a case study, not a runbook. Specific products, version numbers, exact architectural details, IP addresses, hostnames, domain names, rule IDs, script internals, and credential paths have all been deliberately kept out of this writeup. So have narrow category-level descriptions that would narrow a tool choice to one or two likely candidates. The reason is simple: publishing how I think is useful to another engineer, a hiring manager, or an evaluation reviewer; publishing the shape of my specific stack would primarily serve an attacker looking for known-version vulnerabilities. I have kept the methodology and the reasoning and stripped almost everything else.

The short version of what I did. I built layered defensive infrastructure across two physical machines, documented a per-phase threat model, closed every audit finding with a trace from the finding ID to the commit that fixed it, and wrote a set of production monitoring and auto-healing scripts that keep the whole thing running while I am at class. The companion PDFs in this repository describe the methodology, the threat model, the script design patterns, and the integration lessons learned, in each case at a level of abstraction that a reader can learn from without extracting my configuration.

2 Why I built it

Mr. Robot shaped how I think about operating online: compartmentalize everything, trust nothing you cannot verify, and make sure the tools you use are things you actually understand. The project name is a nod to the fsociety collective in the show. The substance, though, is not about mimicking a TV series; it is about building the things a security professional actually needs to understand, in a setting where getting it wrong has consequences I have to live with.

The other reason is practical. I am a Computer Science student at the University of Lethbridge and I plan to work in security. The certifications I want (CompTIA Security+, then OSCP) are on my roadmap but I have not earned them yet. The fastest path I know to turn that gap into credibility is to build something real, document it rigorously, and make the documentation legible to someone evaluating my work. That is what this repository is.

3 Architecture, at an abstract level

The system is physically compartmentalized across two machines. One is always on and runs the infrastructure services. The other is on-demand and runs the operations work. The rationale is that if the operations side were ever compromised, the compromise would not bleed into the secrets store, the password vault, or the daily network. The two trust zones share nothing but a switch port.

The logical network is segmented into multiple trust zones. There is a zone for everyday home use, a zone for self-hosted services and their admin plane, a zone for anonymity-focused activity, and a zone for offensive learning. Each zone has its own policy. The offensive zone can reach the internet for training labs but cannot reach any of the other zones; a mistake in a CTF box cannot become a mistake on my password vault.

I am not including a more detailed network diagram or zone map in this public writeup. The exact segmentation, inter-zone rule structure, and service placement are the kind of detail that is useful primarily to someone trying to target the system, and they are not required to evaluate whether the design is reasonable.

4 Tools

I am not listing the tools I used, at any level of specificity. A public category-by-category breakdown is still narrow enough that a reader familiar with the field can often reduce each slot to one or two likely products, and the combination of those reductions is a plausible stack fingerprint that I would rather not put on the internet next to my real name.

What is happening inside the box is the usual stuff. There is packet filtering at the perimeter and between zones, there is detection that looks at traffic and at hosts, there is encrypted remote access for devices I own, there is identity-gated access to anything I care about, there is a place for secrets that is not on any individual machine, there is a layered path for name resolution, and there is a logging backbone that the detection layer watches. Each slot is filled by an open-source or community-maintained tool that was chosen against at least one alternative, with the reasoning documented in my private planning directory.

If you are reading this to evaluate whether the design is reasonable, what matters is the shape of the whole, not which specific product fills each slot. If you are reading this because you build something similar and you are curious which tool I picked for a given layer, I am happy to talk in a non-public channel.

5 Defensive bias of the roadmap

The roadmap is deliberately weighted toward defensive work. The majority of the phases harden something, add detection somewhere, reduce the attack surface of some component, or improve the resilience of the system. A smaller set are hybrid: writing detection requires understanding the attack it is meant to detect, so those phases have a defensive purpose even when they involve read-

ing offensive material. A smaller still set are offensive learning phases: recon, web exploitation, binary exploitation, anonymity operations, physical security assessment, and an advanced track aimed at OSCP preparation.

This bias is intentional. Defense-first does not mean offense-last; it means offense *compartmentalized*. The offensive phases run inside a segregated zone with no network path to the rest of the infrastructure. If something I am practicing against in a CTF box ever gets out of the CTF box, the blast radius stops at one zone.

6 How the project is organized

The project lives in a private planning directory with one document per phase and a master orchestrator at the top. Every session I start by reading the orchestrator, finding the current phase, and working through it step by step. Each phase has a research section, a decision log, a pre-flight checklist, an execution plan, a verification checklist, and a post-mortem note. The planning is written before the configuration is.

This is deliberate, because the cost of a bad change on a live system is significant. Early in the project I sequenced a dangerous reconfiguration before the safety net was in place and nearly caused a multi-hour internet outage. The refined phase-ordering principle that came out of that incident is in the companion document on hardening methodology. The short version: safe changes first, then reversible changes, then dangerous changes with a dead-man SSH session open, then cosmetic cleanup. Every phase is gate-checked; an unchecked box blocks the phase from starting.

7 Status

I am not publishing the granular phase status because the phase IDs themselves would help someone trying to reverse-engineer what I have and have not hardened yet. At a high level: the foundational network work is complete; the first pass of hardening is complete; a second pass is in progress; the offensive learning phases are planned and have not started.

8 Companion documents

The rest of the case study is split across five documents. Read them in order.

1. 2-ThreatModel.pdf, which sets up what I defend against, what I explicitly do not, and the assume-breach scenarios I gate every critical change on.
2. 3-HardeningMethodology.pdf, which describes the finding-with-traceability audit loop and the rule for how I sequence changes.
3. 4-ScriptsShowcase.pdf, which describes the production monitoring automation at the design-goal level, without code and without layer-specific mechanism.

4. 5-LessonsLearned.pdf, which collects the integration traps I hit, generalized to the class-of-trap level.
5. 6-Security.pdf, which states the redaction posture of this repository and explains how to tell me if you find a leak.

If you are reading to evaluate my work, I would start with 2-ThreatModel and 5-LessonsLearned. They are the two documents that most directly show how I think.